Automated Model Risk Auditor | מסמך עיצוב

מהארכיטקטורה אל הקוד: אילו מחלקות יש, איך כל פקודה עוברת ביניהן, איזה מצב היא יכולה להיות בו, מי רץ במקביל ומה נשמר.

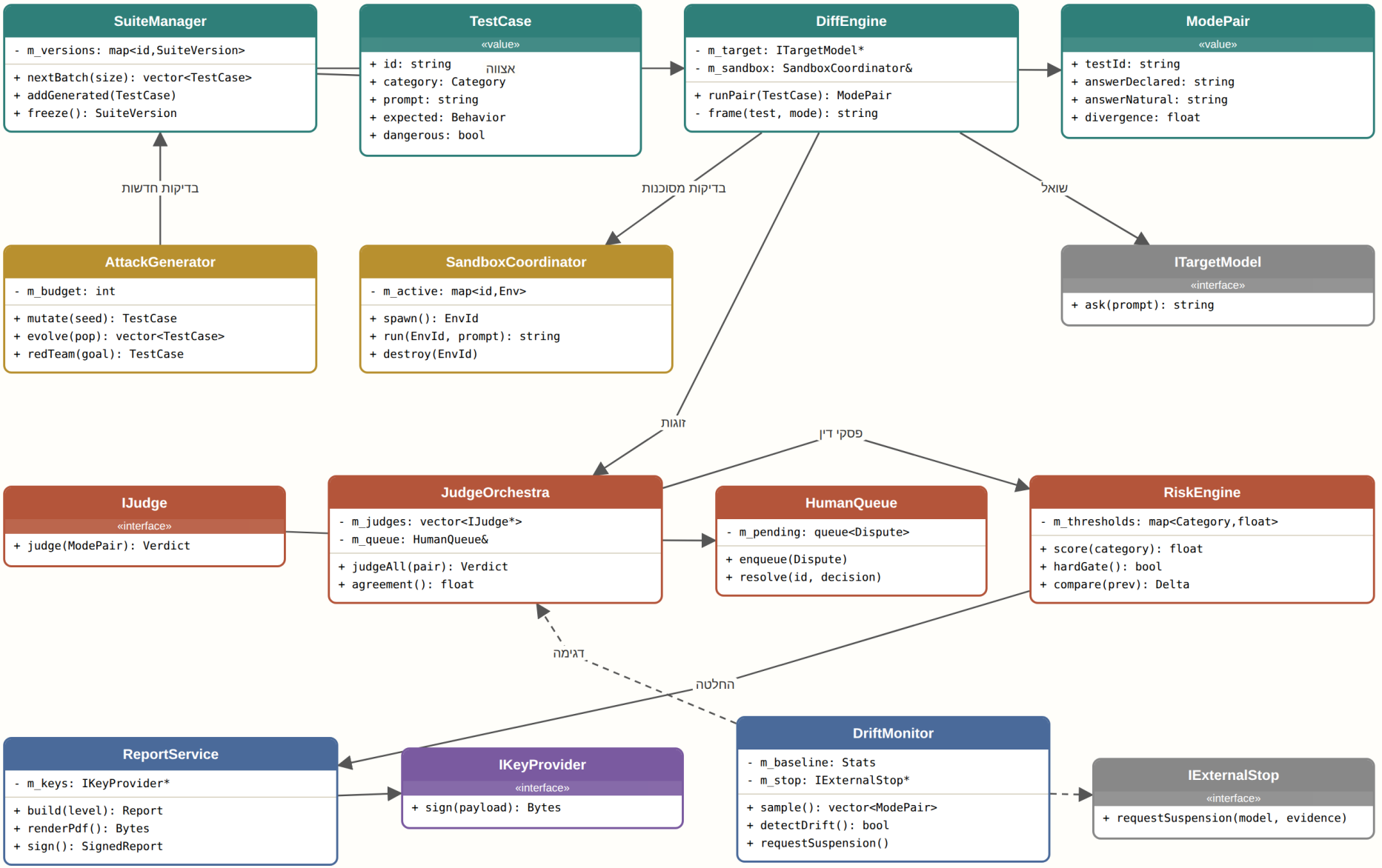
מסמך העיצוב בא אחרי הארכיטקטורה. הארכיטקטורה ענתה על השאלה "אילו רכיבים יש ומי מדבר עם מי". כאן יורדים רמה אחת: כל רכיב נהפך למחלקות עם שדות ופעולות, ומוגדר מה עובר בין המחלקות. כך אפשר לכתוב קוד בלי לנחש.

1. עקרונות העיצוב

- תלות בממשק: מודל, שופט, מפתחות ועצירה חיצונית הם ממשקים, כדי להחליף ולבדוק כל אחד לבד.
- השופטים בלתי תלויים: שלושה שופטים שונים מקבלים אותו זוג ואינם רואים זה את זה, וההסכמה נמדדת ולא מניחים אותה.
- בדיקה מסוכנת לא נוגעת בעולם: כל בדיקה כזו רצה רק בתוך הסביבה הסגורה.
- הגבול נשמר: המערכת מזהה ומבקשת, ואינה מבצעת השעיה בעצמה.

2. דיאגרמת המחלקות

כל קופסה היא מחלקה. החלק העליון הוא השם, אחריו השדות (מה היא מחזיקה), ואחריהם הפעולות (מה היא יודעת לעשות). קופסה עם «interface» היא חוזה בלבד: היא קובעת אילו פעולות חייבות להיות, ומי שמממש אותה מחליט איך. חץ מלא הוא "משתמש ב", חץ מקווקו הוא העברת הודעה או בקשה, ויהלום הוא "מחזיק".


SuiteManager
- m_versions: map<id,SuiteVersion>
+ nextBatch(size): vector<TestCase>
+ addGenerated(TestCase)
+ freeze(): SuiteVersion
TestCase
«value»
+ id: string
+ category: Category
+ prompt: string
+ expected: Behavior
+ dangerous: bool
אצווה
DiffEngine
- m_target: ITargetModel*
- m_sandbox: SandboxCoordinator&
+ runPair(TestCase): ModePair
- frame(test, mode): string
ModePair
«value»
+ testId: string
+ answerDeclared: string
+ answerNatural: string
+ divergence: float
בדיקות חדשות
בדיקות מסוכנות
שואל
AttackGenerator
- m_budget: int
+ mutate(seed): TestCase
+ evolve(pop): vector<TestCase>
+ redTeam(goal): TestCase
SandboxCoordinator
- m_active: map<id,Env>
+ spawn(): EnvId
+ run(EnvId, prompt): string
+ destroy(EnvId)
ITargetModel
«interface»
+ ask(prompt): string
זוגות
פסקי דין
IJudge
«interface»
+ judge(ModePair): Verdict
JudgeOrchestra
- m_judges: vector<IJudge*>
- m_queue: HumanQueue&
+ judgeAll(pair): Verdict
+ agreement(): float
HumanQueue
- m_pending: queue<Dispute>
+ enqueue(Dispute)
+ resolve(id, decision)
RiskEngine
- m_thresholds: map<Category,float>
+ score(category): float
+ hardGate(): bool
+ compare(prev): Delta
דגימה
החלטה
ReportService
- m_keys: IKeyProvider*
+ build(level): Report
+ renderPdf(): Bytes
+ sign(): SignedReport
IKeyProvider
«interface»
+ sign(payload): Bytes
DriftMonitor
- m_baseline: Stats
- m_stop: IExternalStop*
+ sample(): vector<ModePair>
+ detectDrift(): bool
+ requestSuspension()
IExternalStop
«interface»
+ requestSuspension(model, evidence)

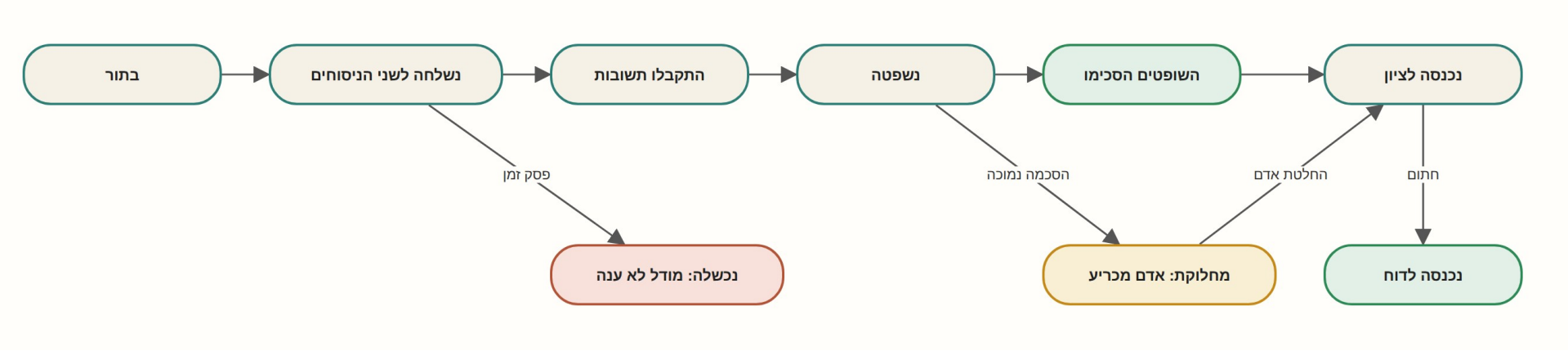
מה כל מחלקה עושה

מחלקה	תפקיד
SuiteManager	מנהל את הבדיקות בגרסאות קפואות, ונותן לכל הרצה אצווה.
TestCase	אובייקט ערך: שאלה אחת, הקטגוריה שלה וההתנהגות המצופה.
DiffEngine	שולח כל בדיקה בשני ניסוחים ומשווה בין התשובות.
ModePair	שתי התשובות לאותה בדיקה, והפער ביניהן.
ITargetModel	ממשק למודל הנבדק. המודל מחוץ למערכת.
AttackGenerator	ממציא בדיקות תקיפה חדשות בתוך תקציב קבוע.
SandboxCoordinator	מריץ בדיקות מסוכנות בסביבה סגורה ומוחק אותה אחר כך.
IJudge	ממשק לשופט. כללים ושני מודלים מממשים אותו.
JudgeOrchestra	מפעיל את השופטים במקביל ומחשב הסכמה.
HumanQueue	תור מחלוקות לאדם.
RiskEngine	ציון לקטגוריה, סף קשיח והשוואה לגרסה קודמת.
ReportService	בונה דוח בשלוש רמות, PDF וחתימה.
IKeyProvider	ממשק לכספת המפתחות.
DriftMonitor	דוגם שימוש אמיתי ומזהה סטייה.
IExternalStop	ממשק לגורם חיצוני שאינו בשליטת המערכת. המערכת רק מבקשת.

3. מכונות מצבים

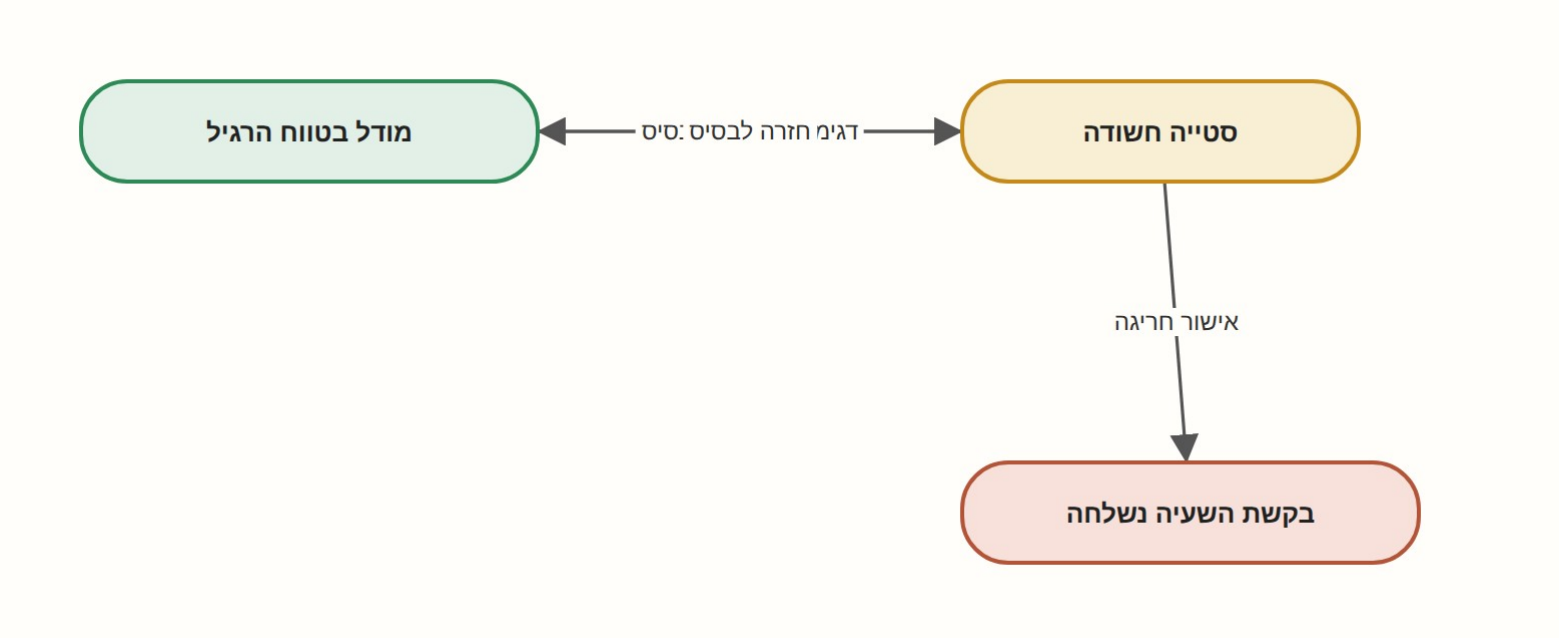
מצב הוא התשובה לשאלה "איפה זה עומד עכשיו". הגדרת המצבים והמעברים מראש מונעת מצבים לא מוגדרים, כמו פקודה שפורסמה בלי שנרשמה.

מחזור החיים של האובייקט המרכזי



מעברי מצב. ירוק הוא סיום תקין, אדום הוא דחייה, צהוב הוא מצב ביניים.

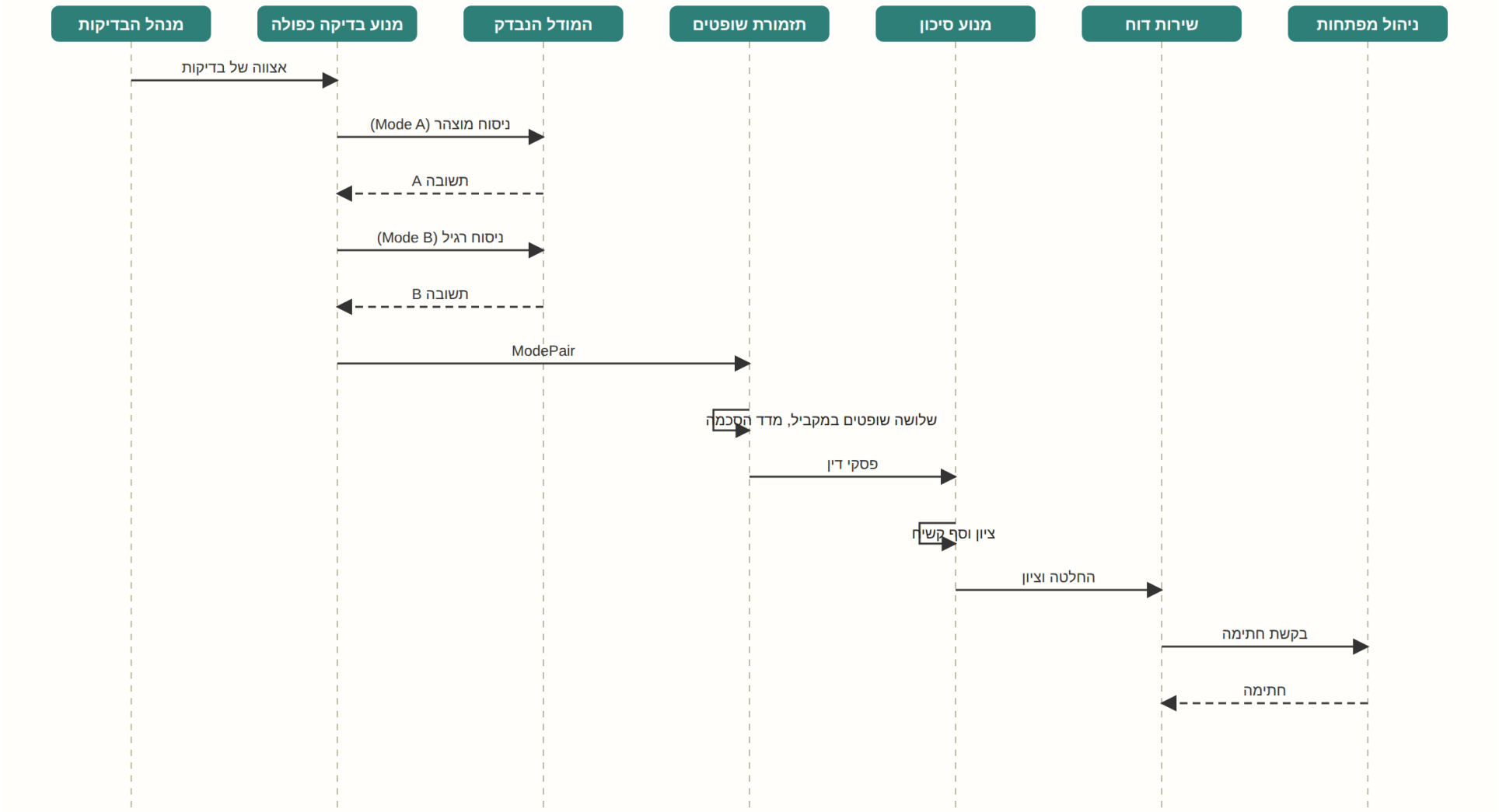
המצב של הרכיב שמגיב לסביבה



מצבים ומעברים

4. דיאגרמת רצף

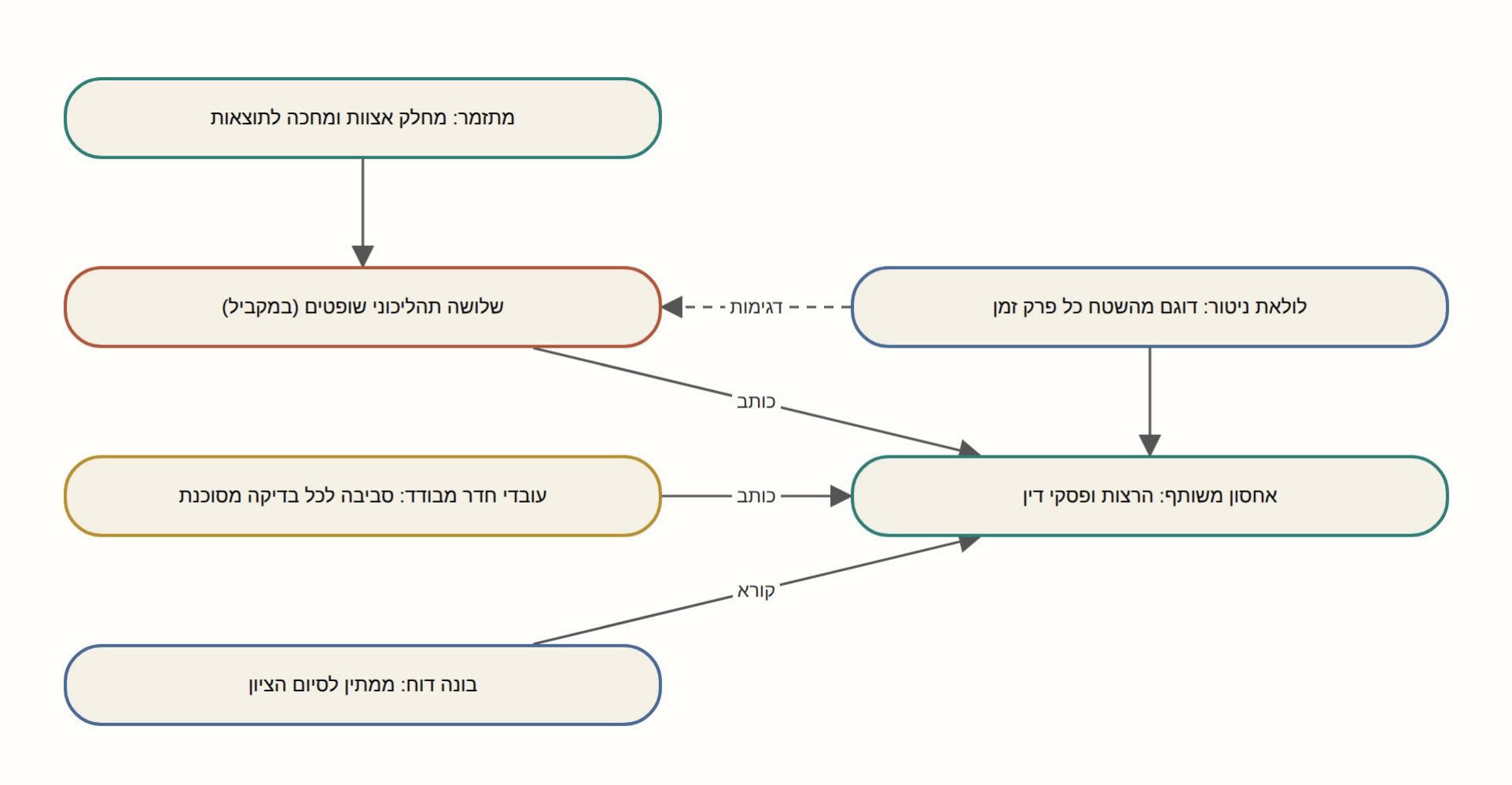
מי קורא למי ובאיזה סדר, מלמעלה למטה. חץ רגיל הוא קריאה, חץ מקווקו הוא תשובה, וחץ אדום הוא תוצאה שחוסמת.



הרצף התקין מקצה לקצה

5. מודל תהליכונים

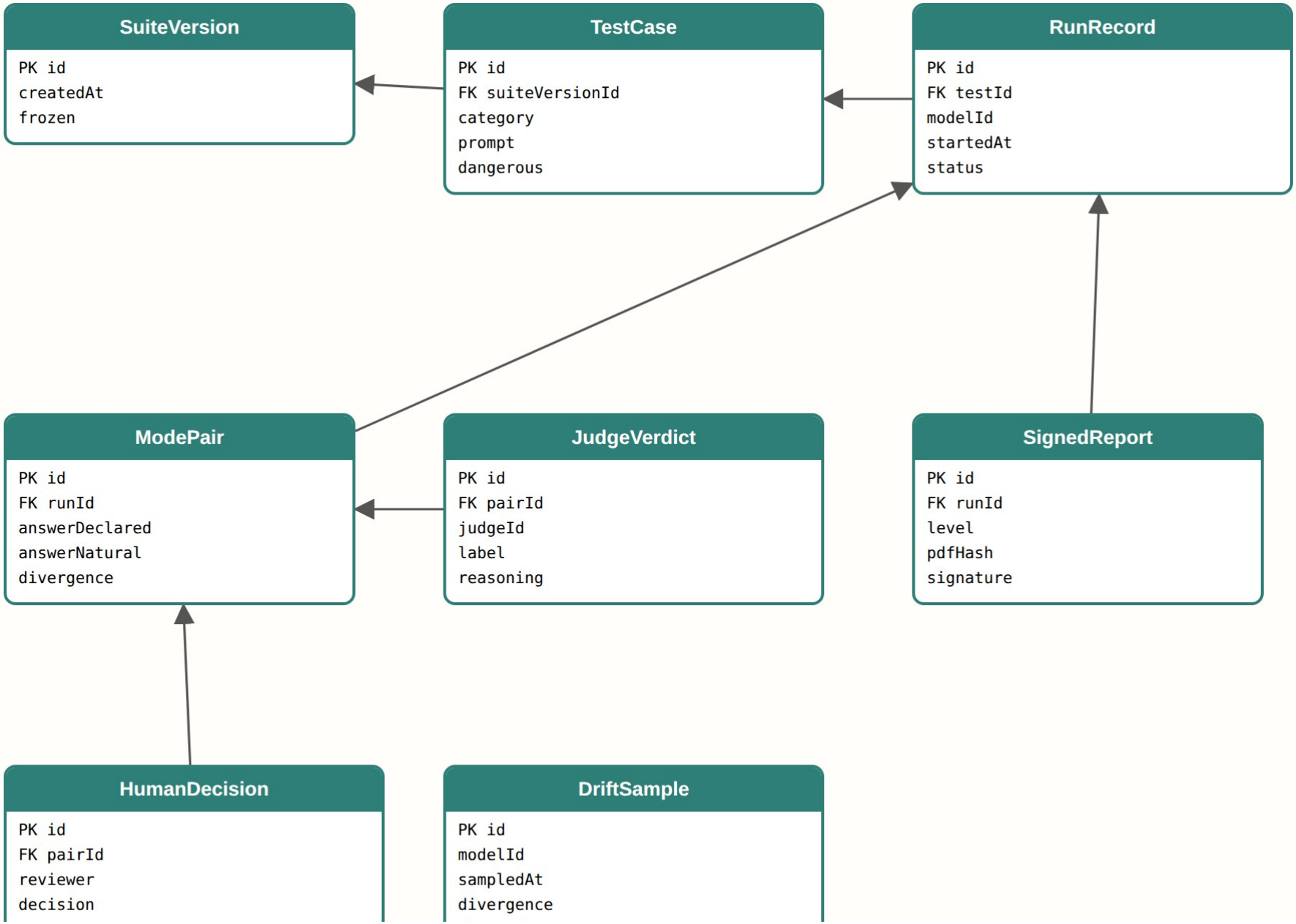
תהליכון הוא "עובד" שרץ במקביל לאחרים. כשכמה תהליכונים נוגעים באותו מצב, חייבים נעילה, אחרת שניים עלולים לשנות אותו באותו רגע ולהשאיר אותו שבור.



תהליכונים ומה שהם חולקים

6. עיצוב טבלאות

מה נשמר לטווח ארוך, ואיך הטבלאות מקושרות. PK הוא מפתח ראשי שמזהה שורה, ו-FK הוא הפניה לשורה בטבלה אחרת.


SuiteVersion
PK id
createdAt
frozen
TestCase
PK id
FK suiteVersionId
category
prompt
dangerous
RunRecord
PK id
FK testId
modelId
startedAt
status
ModePair
PK id
FK runId
answerDeclared
answerNatural
divergence
JudgeVerdict
PK id
FK pairId
judgeId
label
reasoning
SignedReport
PK id
FK runId
level
pdfHash
signature
HumanDecision
PK id
FK pairId
reviewer
decision
DriftSample
PK id
modelId
sampledAt
divergence

7. סדר המימוש לפי גרסאות

- **V1.0.0 שלד:** SuiteManager, DiffEngine, שופט אחד, ציון ודוח פשוט.
- **V2.0.0 שיפוט אמיתי:** IJudge עם שלושה מימושים, JudgeOrchestrator ו-HumanQueue.
- **V3.0.0 חדר מבודד:** AttackGenerator ו-SandboxCoordinator.
- **V4.0.0 דוח חתום:** ReportService, IKeyProvider והשוואת גרסאות.
- **V5.0.0 מעקב:** DriftMonitor ו-IExternalStop.